# Hospital Management System (HMS) Project Report

## Week 1: Planning and Database Design

### Lab Tasks

1. **Project Selection**: The project chosen was a Hospital Management System (HMS) to streamline hospital operations, including patient records, appointments, billing, and inventory management.
2. **Requirements and Features Definition**:
   - **Objective**: Develop a web-based HMS to manage patient records, appointments, consultations, billing, inventory, and staff details.
   - **Features**:
     - Patient registration and medical history tracking.
     - Appointment scheduling with reminders.
     - Billing and insurance claim processing.
     - Inventory management for medical supplies.
     - Role-based access for admins, doctors, and patients.
     - Data security and compliance with regulations (e.g., HIPAA, GDPR).
   - **User Roles**:
     - **Admins**: Manage appointments, registrations, billing, and inventory.
     - **Doctors**: Access patient records, diagnose, prescribe treatments, and update consultation details.
     - **Patients**: Book appointments, view medical records, and pay bills.
3. **Database Schema Design**:
   - **Tables and Relationships**:
     - Tables: `patients`, `doctors`, `appointments`, `consultation`, `billing`, `inventory`, `department`, `admin`.
     - Relationships:
       - One-to-Many: A doctor can attend many appointments; a patient can book multiple appointments.
       - One-to-One: Each appointment results in one consultation.
       - Foreign Keys: `PatientID` in `appointments` links to `patients`, `DoctorID` in `appointments` links to `doctors`.
   - **Entity-Relationship Diagram (ERD)**:
     - Created an ERD to visualize relationships between entities like patients, doctors, appointments, and departments.
   - **Attributes and Data Types**:
     - Example: `patients` table includes `PatientID` (int, PK), `Name` (varchar), `Age` (int), `Gender` (enum), `Email` (varchar), `Password` (varchar), `PhoneNumber` (varchar), `Address` (text), `CNIC` (varchar).
     - Constraints: Primary keys, foreign keys, `NOT NULL` for critical fields, `UNIQUE` for identifiers like `Email` and `CNIC`.

# Week 1: Front-End Development

1. **User Interface (UI) Design**:
   - Designed using HTML and CSS to create an intuitive and user-friendly interface.
   - Pages included: `appointment`, `billing`, `consultation`, `dashboard_admin`, `dashboard_doctor`, `dashboard_patient`, `doctor_profile`, `inventory_management`, `login`, `patient_form`, `reports`.
2. **Web Pages for Functionalities**:
   - Created pages for listing, adding, editing, and deleting records (e.g., patient registration, appointment scheduling).
3. **Responsive Design**:
   - Implemented CSS frameworks (e.g., Bootstrap) to ensure compatibility across mobile and desktop devices.
4. **JavaScript Implementation**:
   - Added client-side validation for forms (e.g., ensuring valid email and phone number formats).
   - Incorporated interactivity for features like dynamic appointment scheduling and real-time form feedback.

# Week 2: Back-End Development and Database Integration

1. **PHP Scripts Development**:
   - Developed PHP scripts to handle user input, process requests, and interact with the MySQL database.
   - Implemented CRUD operations:
     - **Create**: Add new patients, appointments, and inventory items.
     - **Read**: Retrieve patient records, appointment schedules, and billing details.
     - **Update**: Modify patient information, consultation details, and inventory quantities.
     - **Delete**: Remove outdated appointments or inactive patient records.
2. **Database Connection**:
   - Configured database connection settings in PHP to connect to the MySQL database (`project.db`).
   - Created reusable functions for database interactions (e.g., `insertPatient()`, `retrieveAppointments()`, `updateBillingStatus()`, `deleteInventoryItem()`).
3. **Testing**:
   - Verified accurate storage and retrieval of data (e.g., patient records, appointment details).
   - Handled errors such as duplicate entries, invalid inputs, and database connection failures.

# Week 2: Testing, Debugging, and Refinement

1. **Thorough Testing**:

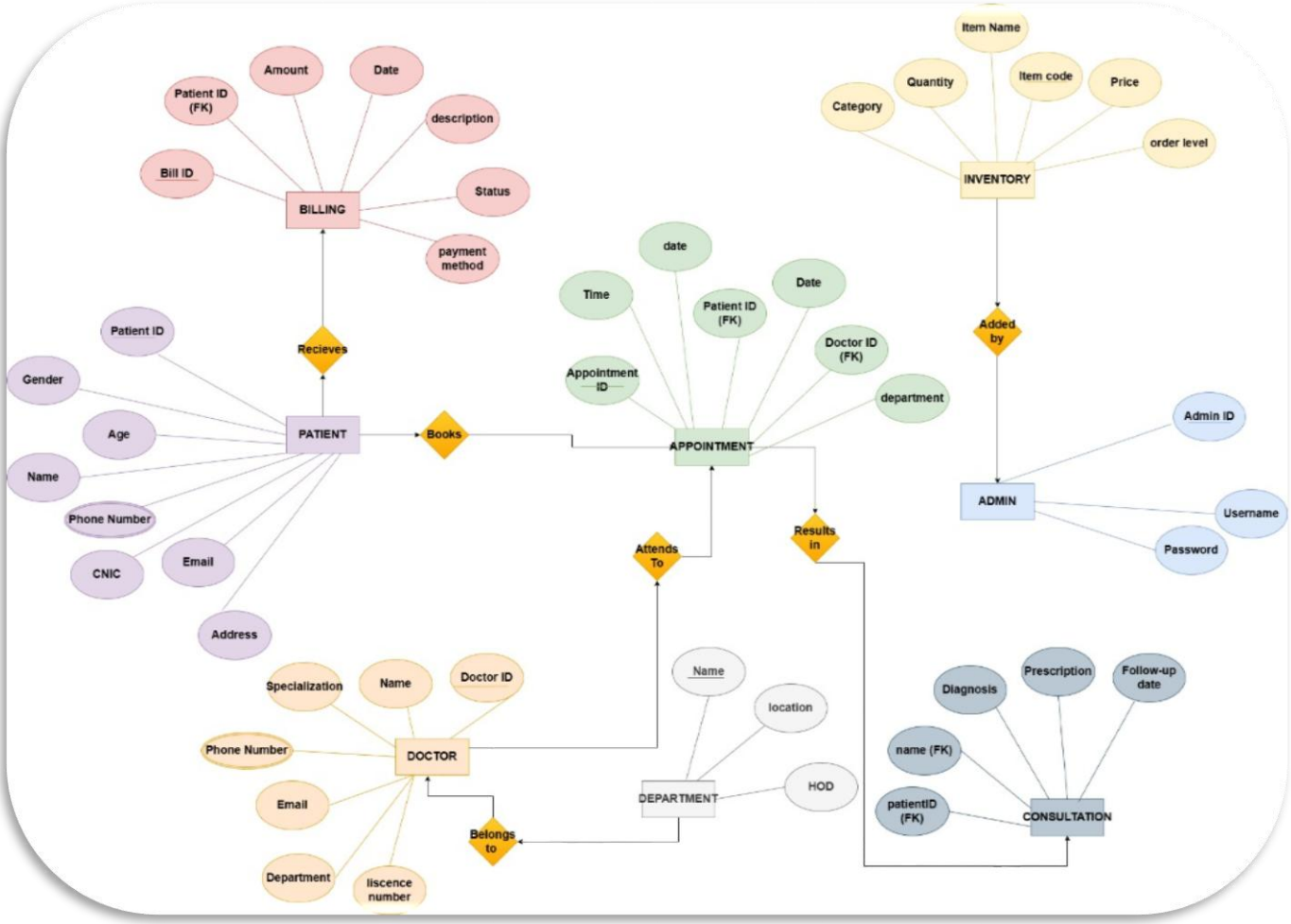
    - Tested all CRUD operations to ensure functionality (e.g., booking an appointment, generating a bill).
    - Validated data integrity (e.g., no negative inventory quantities, correct appointment-doctor linkage).
2. **Debugging**:
    - Fixed issues like incorrect foreign key references and validation errors in forms.
    - Resolved database query errors (e.g., handling `NULL` values in `appointments` table).
3. **Additional Features**:
    - Added search functionality to filter patients, appointments, and inventory items.
    - Implemented user authentication for secure access based on roles (admin, doctor, patient).
4. **Code Optimization**:
    - Optimized database queries for faster retrieval (e.g., indexing frequently accessed columns like `PatientID`).
    - Refactored PHP code to improve readability and reduce redundancy.

# Week 3: Documentation and Presentation

1. **Documentation**:
    - **User Manual**: Detailed guide for users (admins, doctors, patients) on how to navigate and use the HMS (e.g., booking appointments, viewing medical records).
    - **Database Schema Documentation**: Included table structures, relationships, constraints, and sample data from `project_db.sql`.
    - **Code Documentation**: Commented PHP, HTML, CSS, and JavaScript code to explain functionality and logic.
2. **Challenges and Solutions**:
    - **Challenge**: Handling `NULL` values in the `appointments` table (e.g., `PatientID` in some records).
        - **Solution**: Added validation to ensure appointments are linked to valid patients or marked as open slots.
    - **Challenge**: Ensuring data security for sensitive patient information.
        - **Solution**: Implemented password hashing (e.g., bcrypt for patient passwords) and role-based access control.
    - **Challenge**: Synchronizing front-end and back-end validation.
        - **Solution**: Used JavaScript for client-side validation and PHP for server-side validation to ensure consistency.
    - **Challenge**: Managing inventory stock levels to prevent negative quantities.
        - **Solution**: Added validation logic in PHP to check `Quantity` before updating the `inventory` table.

# Conclusion

The Hospital Management System (HMS) was successfully developed to streamline hospital operations, improve patient care, and enhance efficiency. The system integrates a robust MySQL

database with a responsive front-end and secure back-end, supporting various user roles and ensuring data integrity and security. The project addressed challenges through careful planning, testing, and optimization, resulting in a functional and user-friendly application.

# ER DIAGRAM: